Marwadi University

Software Engineering

Assignment-1

Page No.
Date

CO-1- Software Development Process models

Q Explain the dual role of software as both a product and a vechicle for delivering a product. Discuss how the historical software crisis led to the adoption of systematic software Engineering practices.

Software plays 2 important roles - it acts as both a product and a vehicle for delivering a product

1) Software as a product -
Software is a product because it provides features and services directly to users. It includes programs, data, and documentation used to perform specific tasks. Ex - include MS word, web browsers, mobile applications and banking software.

2) Software as a vehicle for Delivering a Product -
Software also works as a vehicle because it controls or supports other products and systems. Ex - software controls washing machines, cars, medical equipment, robots.

Software Crisis -
During the early development of computers software projects became increasingly large and complex. many projects faced problems such as delays, increased poor quality, difficult maintainence and failure to meet user requirments. which was known as software crisis. To solve these problems, systematic software engineering practices were introduced. These include

STUDY TIME

proper planning, requirement analysis, software design, testing, documentation and maintenance. They help develops produce reliable softwares with in the planned budget and time.

2Q compare the linear sequential (waterfall) model with the Incremental Process modul under what conditions would you choose the RAD model over waterfall?

The waterfall, Incremental and Rapid Application Development RAD models are software development approaches used for different project requirements.

Difference between waterfall & Incremental model

waterfall model	Incremental model
Development follows a fixed sequence of phases	Development occurs in small -er parts called incr ? ements
Requirements are generally decided at the beginning	Requirements and features can be introduced across increments.
working software is usually delivered near the end	working software is delivered in multiple stages
changes are difficult & costly after a phase is completed	changes can be handled more easily between increments.
suitable for projects with clear and stable requirements	suitable for projects that can be divided into smaller fn. parts.

STUDY TIME

Marwadi University

Page No.
Date

when should RAD be chosen over waterfall?
RAD should be preferred when:
1) The project must be completed within a short time
2) users are available to provide regular feedback.
3) Requirements can be developed and refined through prototypes
4) The project can be divided into independent modules
5) Reusable components and suitable development tools are available.

Ex - RAD may be suitable for developing a college event registration application quickly, where users can review prototypes and suggest changes.

3Q Why would the spiral model or evolutionary prototyping model be preferable to waterfall for a high-risk, complex enterprise system with changing requirements?

1) Spiral model -
The spiral model develops software through repeated cycles. Each cycle includes planning, risk analysis, development and evaluation.
It identifies and analyzes risks early.
It allows requirements to change during development.
It supports regular customer feedback.
It is suitable for large, complex, and high-risk projects.
Ex - A large banking system can use the spiral model to evaluate security risks before implementing major features.

STUDY TIME

Page

Marwadi University

Page No.
Date

Marwadi University

Page No.
Date

2) Evolutionary prototyping model-
It this model an initial working version of the software is created and gradually improved using user feedback.
It helps clarify unclear requirments.
users can test early versions of the system.
new features can be added over time
It reduces the chance of dereloping a system that does not meet user needs.

why not waterfall?
The waterfall model expects requirements to be relatively stable before development proceeds. If requirements change significantly, earlier work may need to be repeated, increasing cost and time
the spiral model is useful when risk analysis is the main concern, while Evolutionary Prolotyping is useful when understanding user requirements is the main concern.

CO-2 : Agile Paradigms and Herative Delivery

10 Explian the core values of the Agile manifesto and compare them with conventional heavyweight, plan-driven development methods.

The Agile manifesto is a set of values and principles that guide software development. It focuses on delivering useful software accepting changes and working closely with customes.

STUDY TIME

Marwadi University

Page No.
Date

Four core values of Agile:

1) Individuals and interactions over processes and tools: communication and teamwork are more important than depending only on tools and procedures.
2) working software over comprehensive documentation. A working application is given more importance than excessive documentation, although necessary documentation is still maintained
3) customer collaboration over contract negotiation - Developers work closely with customers to understand their needs and feedback.
4) Responding to change over following a plan - Agile accepts changing requirements, even during later stages of development.

Agile vs Traditional Development.

Agile methodology	Traditional Plan-Driven methodology
Development occurs in short iterations	Development usually follows predefined phases.
customer feedback is continuous	customer involvement may be limited after requirements gathering.
changes are accepted throughout development	changes can be difficult after major phases are completed.
working software is delivered frequently.	Emphasizes detailed planning and documentation.
Emphasizes collaboration and flexibility.	

STUDY TIME

Page

conclusion - Agile is useful when requirements change frequently and regular delivery is needed. Traditional methods are suitable when requirements are stable and detailed planning is important.

2a Describle the core practices of Extreme Programming XP focusing on Pair Programming, Refactoring spike solutions, and CRC cards.

Extreme Programming XP is an Agile software development methodology that focuses on improving software quality, teamwork, and customer satisfaction through frequent feedback and continuous development.

Its important practices are-

1) Pair Programming-

2 programmers work together on the same computer, one writes the code, while the other reviews it and suggests improvements. they can switch roles regularly.

ex - one programmer writes a login function while the other checks the logic and identifies errors.

2) Refactoring-

Refactoring means improving the internal structure of existing code without changing its external behaviour.

It makes the code easier to understand, maintain and modify.

Ex - Removing repeated code by creating a common function.

3) Spike solutions -
A spike solution is a small experiment or temporary program created to understand an unfamiliar technology or solve a difficult technical problem. It helps developers estimate the effort and choose an approach before full implementation.
Ex - Creating a small test program to check whether a payment API works with the application.

4) CRC cards -
CRC stands for class, Responsibility and collaborator. CRC cards help developers identify classes, their responsibilities and the other classes they interact with.
class - name of the class
Responsibility - tasks performed by the class
Collaborator - other classes that help it perform those tasks.
Ex - An order class may be responsible for calculating the order total and may collaborate with a payment class.
These practices help XP teams develop maintainable software, identify errors easily, and improve collaboration.

Marwadi University

Page No.
Date

39 Define the key roles in scrum & explain the workflow from managing the product Backlog to sprint planning and daily stand-ups.

scrum is an agile framework in which a team develops software through short, fixed periods called sprints

Key roles in scrum-

1) Product owner- The product owner manages and prioritizes the product Backlog. They communicate customer needs & help ensure that the team works on valuable features.

2) scrum master - the scrum master helps everyone understand, and follow scrum. They support the team remove obstacles, and improve the development process

3) Developers - Developers build test, and deliver a usable product increment during each sprint. They organize their work to achieve the sprint goal.

Scrum workflow-

1) owner maintains a list of required features, improvements and fixes, arranged according to priority.
2) The scrum team discusses the sprint goal, selects suitable backlog items, and plans how to complete them.
3) Developers work on the selected items during the sprint.

STUDY TIME

Page

TIME

4) Developers meet briefly each day to inspect progress toward the sprint goal and adjust their plan.
5) The team presents the completed work and gathers feedback from stakeholders.
6) The team discusses what went well and what can be improved in the next sprint.

CO-3 Requirements and software metrics.

1Q Explain the 4Ps of software management and how Bochm's WSHH principle helps establish project characteristics.

The 4Ps of software management are People product. These 4 factors help managers organize and control software development.

1) people - people include project managers, developers, testers, customers and other stakeholders.

2) Product - The product refers to the software being developed. Its objectives, features, scope, constraints and requirement must be clearly understood.

3) Process - The process defines the activities and methods used to develop software, such as requirements analysis, design, coding, testing and maintenance.

4) Project - the project includes planning, scheduling, resource allocation, cost estimation and risk management.

STUDY TIME

Page No.
Date

Boehm's WSHH principle-

Boehm's WSHH principle provides 7 questions that helps managers understand and plan a software project.

why? why is the system being developed?
what? what work must be done?
when? when will the project be completed?
who? who is responsible for each task?
where? where are the resources & people located?
How? How will the work be performed technically and managerially?
How much? How much effort, time and cost are required?

20 Differentiate between Functional and non-functional Requirements. Explain the stages of Requirements Engineering from Inception to validation and management.

Requirement Engineering is the systematic process of identifying analyzing documenting, validating and managing the needs of users and stakeholders.

Functional Req.	non-functional Req.
Describe what system must do	Describe how well the system must perform
Define features and services	Define quality attributes & limitations.
Related to system behaviour	Related to performance, security, usability etc
Ex- A user can reset a password	Ex- the password reset page loded in seconds.

STUDY TIME

Page

stages of Requirment engineering-
1) Inception- understand the problem & project goals
2) Elicitation- collect requirments from users
3) Elaboration - Analyze and refine requirments
4) Negotiation- Resolve conflicts and set priorities
5) specification- Document requirements in an SRS
6) validation- check whether requirment are correct and complete.
7) management - Track & control requirement chang es.

3Q Differentiate measures, metrics, and indicators Explain LOC, MTBF, MTTR & MTTC.

measure A directly collected value, such as 2000 lines of code.
metric - A measurement used to evaluate software such as defects per 1000 lines.
Indicator - Information used to understand project status and support decisions.

Types of metrics-
process metrics- measure development activities such as testing efficiency.
Product metrics - measures software characteristics such as size and reliability.

LOC - lines of code measures program size
MTBF - mean time between Failures average operating time between failures.
MTTR - mean time to repair, average time to restore a failed system.

MTTC. - mean time to change avarage time needed to implement a change.

CO-4 Project scheduling and effort Estimation

1Q Explain the 5 dimensions of a feasibility study & the importance of scope management.

A feasibility study checks whether a software project is practical.

1) Technical Feasibility - checks the availibility of technology and skills.
2) Operational Feasibility - checks weather the system meets user and organizational needs.
3) Economic Feasibility - compare project costs with expected benefits.
4) Legal feasibility - checks compilance with laws and regulations
5) schedule feasibility - checks whether the project can be completed on time.

Importance of scope management - Scope management defines the work included in a project. It prevents unnecessary features, known as scope creep from increasing costs and delays. It also helps manage resources and meet project objective.

2Q Explain the cocomo model calculate the effort and development time for a 300 KLOC organic project using basic cocomo.

Marwadi University

Cocomo stands for constructive cost model. It is a software cost estimation model used to calculate the effort and development time required for a software project based on its size.

The basic cocomo model classifies projects into 3 types -
organic - small and relatively simple projects developed by experienced teams.
semi-detached - medium sized projects with moderate complexity.
Embedded - complex projects with strict hardware, software, or o.s.

Given project size = 300 KLOC
project type = organic

$E = 2.4 \times (KLOC)^{1.05}$
$Tdev = 2.5 \times (E)^{0.38}$

$E = 2.4 \times 300^{1.05}$
$E = 957$ person-months

$Tdev = 2.5 \times (957)^{0.38}$
$Tdev = 35.9$ months

3a Compare PERT and CPM. Explain how WBS and Gantt charts help project managers track critical paths and allocate resources.

STUDY TIME

Marwadi University NAAC

Page No.
Date

PERT and CPM are project scheduling techniques used to plan, organize, and monitor project activities.

Difference between PERT and CPM-

PERT	CPM
stands for Program Evaluation and Review Technique	stands for critical path method.
Uses three time estimate optimistic, most likely and pessimistic.	Traditionally uses a single estimated duration for each activity.
Suitable when activity durations are uncertain	suitable when activity durations are reasonably predictable.
Focuses on handling time uncertainty.	Focuses on scheduling and time-cost optimization
commonly used in research and development projects	commonly used in construction and planned development projects.

Work Breakdown structure (WBS)-
A WBS divides a large project into smaller, manageable tasks. It helps managers identify all the work required, assign responsibilities, estimate costs and allocate resources.

Gantt chart-
A gantt chart is the ~~longest~~ duration a visual schedule that displays project activities as horizontal bars against a timeline. It shows task ~~tar~~ start dates end dates durations and overlapping activity

STUDY TIME

Page

Marwadi University | NAAC A+

Page No.
Date

Critical path-

The critical path is the longest-duration path through the project activity network. It determines the minimum time required to complete the project, assuming the activity estimates and dependencies are correct.

CO-5 Software Risk management

1Q Differentiate between Proactive and Reactive risk strategies categorize software risks into project, Technical, Business and management risks with examples.

Risk management is the process of identifying possible problems that may affect a software project and taking steps to reduce their impact.

Proactive vs Reactive Risk strategies

Proactive strategy	Reactive strategy
Identifies risks before they occur	Responds after a risk becomes a problem
Focuses on prevention and preparation	Focuses on controlling damage and recovery
Includes risk analysis and contingency planning	Includes emergency fixes and corrective actions
Helps reduce unexpected problems	may result in additional cost and delays.

Ex - Assigning a replacement after a developer leaves.

STUDY TIME

Type of software Risks

1) Project Risk - Affects schedule, budget, or resources. Ex - A developer leaving causes delays.
2) Technical Risk - Relates to technology or software quality. Ex - the database cannot handle expected traffic
3) Business Risk - Affects commercial success. Ex - A competitor launches a similar product
4) management Risk - Results from poor planning or coordination Ex - unclear responsibilities cause repeated work.

2Q Explain the RMMM Plan and the fields of a Risk Information sheet.

RMMM stands for Risk mitigation, monitoring and management. It describes how project risks will be handled.

main parts of RMMM -

1) Risk mitigation - Actions to reduce the probability or impact of a risk.
2) Risk monitoring - Regularly checking risks and warning signs
3) Risk management - Taking action when a risk occurs.

Risk information sheet (RIS) records details about an individual risk. Its main fields are:

Risk ID - Unique identification number
Risk Description - Explanation of the risk.
Risk category - type of risk
Probability - chance of occurrence.
impact = Possible effect on the project
Priority - Importance of the risk.
mitigation strategy - Preventive actions
monitoring Indicators - warning signs.
contingency Plan - Actions to take if the risk occurs.
Risk owner - Person responsible for managing the risk.

3Q Differentiate Software Reliability and software Safety. Explain hazard analysis.

Software Reliability - The ability of software to operate without failure for a specified period under specified conditions

software safety - The ability of a system to avoid causing unacceptable harm.

Reliability	Safety.
Focuses on failures	Focuses on hazards and harm
Evaluted using MTBF and reliability testing	Evaluated using hazard and risk analysis.
Ex - Banking software works without. unexpected failure	Ex - medical equipment prove nts an unsafe dosage.

steps in hazard Analysis-

1) Identify Hazards- Find conditions that may cause harm.
2) Identify causes- Determine possible causes, such as sensor failure or incorrect code.
3) Assess Risk- Evaluate the likelihood and severity of harm.
4) Define safety Requirements- specify rules to prevent dangerous situations.
5) Implement safeguards- Add alarms, emergency shut-downs, or safety controls.
6) Test and verify- check that safety requirements work correctly.
7) monitor- Review safety when the system changes.

STUDY TIME